# Queue LESS web-based food ordering

**A MINI PROJECT REPORT**

Submitted to the

CS2307 – Internet Programming Laboratory

Submitted by

A Priyadarsan

(24UAD037)

M Benedict Joshua

(24UAD009)

Dinesh Bharathy A

(24UAD012)

of

**KAMARAJ COLLEGE OF ENGINEERING AND TECHNOLOGY**




**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

**KAMARAJ COLLEGE OF ENGINEERING AND TECHNOLOGY**

**(An Autonomous Institution - Affiliated to Anna University, Chennai)**

**K.VELLAKULAM, VIRUDHUNAGAR - 625 701**

**October-2026**

## TABLE OF CONTENTS


| CHAPTER No. | TITLE | PAGE No. |
|---|---|---|
| | **ABSTRACT** | **3** |
| **I** | **INTRODUCTION** | **4** |
| | 1.1 Overview | **5** |
| | 1.2 Motivation | **5** |
| | 1.3 Importance | **6** |
| | 1.4 Objectives | **7** |
| **II** | **SYSTEM REQUIREMENT SPECIFICATION** | **8** |
| | 2.1 Hardware specification | **10** |
| | 2.2 Software specification | **10** |
| **III** | **SYSTEM DESIGN** | **11** |
| | 3.1 Database design | **12** |
| | 3.1.1 ER Diagram | **13** |
| | 3.1.2 Normalization | **14** |
| | 3.1.3 Data dictionary | **16** |
| | 3.2 High Level Design | **19** |
| | 3.3 Detailed Design | **20** |
| **IV** | **IMPLEMENTATION** | **24** |
| | **4**.1 Deployment Procedure | **25** |
| | 4.2 Coding | **26** |
| | 4.3 Screenshots | **30** |
| **V** | **CONCLUSION AND FUTURE ENHANCEMENT** | **39** |

## ABSTRACT

The QueueLess is a web-based food ordering and token management application designed to provide an efficient, convenient, and user-friendly solution for managing food orders in college canteens. In many educational institutions, students have to wait in long queues to place their orders and collect their food, especially during peak hours. Manual ordering and queue management can lead to delays, overcrowding, confusion, and difficulty in tracking orders. The proposed system provides a centralized digital solution that allows students to order food in advance, receive a unique token number, and collect their order when it is ready.

The system provides separate access for two types of users: Student and Admin. Students can log in or create an account, view the available items in the today's menu, select the required quantity, choose a pickup time, and place their order online. After placing an order, the system automatically generates a token number for the student. Students can view their active and previous tokens along with the ordered items, total amount, pickup time, order date, and current order status. Students can also cancel an order while it is still in a cancellable stage.

The administrator uses a dedicated kitchen dashboard to manage the food ordering process. The admin can view today's orders along with the token number, student name, ordered items, pickup time, total amount, and current status. The order status can be updated as the food progresses through different stages such as Placed, Preparing, Ready, and Completed. The updated status is reflected on the student's My Tokens page, allowing students to know when their food is ready for collection and reducing unnecessary waiting at the canteen.

The application is developed using Java, JSP, Servlets, HTML, CSS, and JavaScript, with Apache Tomcat 9 used as the web server and MySQL/MariaDB used for database management. The Java Servlet technology handles user authentication, food ordering, token generation, order cancellation, and status management, while JSP pages provide the user interface for students and administrators. The database stores user details, menu items, orders, and individual order items, providing persistent and organized storage for the application. The application is designed with a responsive and user-friendly interface to make the ordering and order-management process simple and accessible.

The main objective of this project is to reduce waiting time and overcrowding in college canteens, minimize manual order management, and improve communication between students and kitchen administrators. By providing advance ordering, automatic token generation, real-time order status updates, and digital order tracking, QueueLess creates a more organized and transparent food ordering experience. Overall, the QueueLess system provides a simple, efficient, and practical approach to modernizing the college canteen ordering process and improving the experience for both students and canteen administrators.

# CHAPTER I

# INTRODUCTION

# CHAPTER I

## INTRODUCTION

The QueueLess is a web-based food ordering and token management application designed to simplify the food ordering process in a college canteen. Students can sign in, view the available food items, select the required quantity, choose a pickup time, place orders, and receive a unique token number. The system also allows students to view their active and previous orders and track the current status of their food orders. The administrator can view today's orders, manage the kitchen process, and update the status of each order through a dedicated dashboard.

The system reduces manual ordering, waiting time, and overcrowding in the canteen by keeping food orders and token information in one place. It is developed using Java, JSP, Servlets, HTML, CSS, and JavaScript, with Apache Tomcat used as the web server and MySQL/MariaDB used for database management. The application provides a simple, organized, and transparent way to improve communication between students and the canteen administrator and makes the overall food ordering and token management process more efficient.

## 1.1 OVERVIEW

QueueLess is a web-based food ordering and token management system developed for college canteens. It allows students to view the available menu, select food items, choose the required quantity and pickup time, and place their orders online. After placing an order, the student receives a unique token number that can be used to identify and track the order. The system also provides students with details such as ordered items, total amount, pickup time, and order status.

The administrator can access a dedicated kitchen dashboard to view and manage the day's orders. The order status can be updated as the food is being prepared and made ready for pickup. By maintaining order and token information digitally, QueueLess helps reduce manual work, waiting time, and overcrowding in the canteen. The system provides a simple and organized platform for managing food orders and improving the overall canteen experience.

## 1.2 MOTIVATION

The main motivation for developing QueueLess is to provide a simple, efficient, and organized platform for students to order food and manage their pickup in a college canteen. In traditional canteen ordering, students have to stand in queues, place their orders manually, and wait near the counter until their food is prepared. During peak hours, this can result in long queues, overcrowding, delays, and difficulty in managing multiple orders at the same time.

The proposed system provides a centralized digital platform where students can view the available menu, select food items, choose the required quantity, and select a suitable pickup time. After placing an order, the system generates a unique token number for the student. The student can use the token to identify the order and view important details such as the ordered items, total amount, pickup time, and current order status.

The system also helps the canteen administrator manage food orders more effectively. All orders for the day are displayed in a dedicated kitchen dashboard along with the token number, student name, ordered items, pickup time, total amount, and order status. The administrator can update the status of an order as it moves through the preparation process, allowing students to know whether their order has been placed, is being prepared, or is ready for pickup.

Another motivation for this project is to reduce manual effort and unnecessary waiting time in the canteen. By digitizing the food ordering and token management process, the system reduces the need for students to repeatedly stand at the counter and helps the administrator handle orders in an organized manner. It also keeps order information in one place, reducing confusion and making it easier to manage the daily food orders.

## 1.3 IMPORTANCE

The QueueLess system is important because it provides a digital solution for managing food ordering and token generation in a college canteen. Traditional food ordering methods require students to wait in queues and depend on manual communication with the canteen staff. QueueLess provides a structured platform that makes the ordering process more convenient and organized for students.

The system improves the overall efficiency of the canteen by allowing students to place their orders in advance. Students can view the available food items, select quantities, choose a pickup time, and receive a unique token number. This reduces unnecessary waiting near the counter and helps students manage their time more effectively, especially during busy hours.

QueueLess is also important for the canteen administrator because it provides a centralized view of the day's orders. The administrator can see the token number, student details, ordered items, pickup time, total amount, and current status of each order. This makes it easier to monitor food preparation and manage multiple orders systematically.

Another important aspect of the system is improved communication between students and the canteen. Students can check the current status of their orders instead of repeatedly approaching the counter for updates. The administrator can update the order status as the food moves through the preparation process, providing students with clear information about their orders.

Overall, QueueLess helps reduce queues, waiting time, manual effort, and confusion in the canteen. It provides a simple and organized approach to food ordering and token management, making the process more efficient for both students and the canteen administrator.

## 1.4 OBJECTIVES

The main objective of the QueueLess system is to develop a web-based food ordering and token management application that simplifies the process of ordering food in a college canteen. The system aims to provide students with a convenient platform to view the menu, select food items, place orders, and receive a token without having to depend completely on manual ordering at the canteen counter.

The system aims to reduce the waiting time and overcrowding faced by students during busy hours. By allowing students to place their orders in advance and select a suitable pickup time,

QueueLess helps organize the food collection process and allows students to spend less time waiting in queues.

Another objective is to provide an effective order management system for the canteen administrator. The administrator should be able to view the day's orders along with token numbers, student names, ordered items, pickup times, total amounts, and order statuses. The system should also allow the administrator to update the status of orders during the food preparation process.

The system also aims to provide students with clear information about their orders. Students should be able to view their active and previous tokens, ordered items, total amount, pickup time, date, and current status. This improves communication between the student and the canteen and reduces the need for repeated enquiries about order progress.

Overall, the objective of QueueLess is to create a simple, organized, and user-friendly digital solution that reduces manual work, improves order tracking, minimizes unnecessary waiting, and makes the college canteen food ordering process more efficient for both students and administrators.

# CHAPTER II

# SYSTEM REQUIREMENT SPECIFICATION

# CHAPTER II

# SYSTEM REQUIREMENT SPECIFICATION

The QueueLess system requires suitable hardware and software resources to develop, deploy, and run the web-based food ordering and token management application. The following requirements are necessary for the proper functioning of the system.

## 2.1.1 HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 10 GB free space
- Keyboard and Mouse
- Monitor
- Network connection

## 2.1.2 SOFTWARE REQUIREMENTS

- Operating System: Windows 10/11
- Programming Language: Java
- Front-End: JSP, HTML, CSS, JavaScript
- Back-End: Java Servlets
- Database: MySQL/MariaDB
- Database Management: phpMyAdmin
- Web Server: Apache Tomcat 9
- IDE: Eclipse
- JDK: Java 17
- Database Connectivity: JDBC
- Server Environment: XAMPP

## 2.1 HARDWARE SPECIFICATION

For running our proposed system we need the following hardware requirements for implementation such as,

| | |
|---|---|
| **Processor** | Intel Core i3 or above |
| **RAM** | 4 GB minimum |
| **Hard Disk** | 10 GB free space |
| **Display** | 1366 × 768 resolution or higher |
| **Keyboard and Mouse** | Standard |

## 2.2 SOFTWARE SPECIFICATION

For running our proposed system we need the following software requirements for implementation such as,

| | |
|---|---|
| **Operating System** | Windows 10/11, Linux, or macOS |
| **Code Editor** | Visual Studio Code |
| **Programming Language** | Java |
| **Web Technology** | HTML, CSS, Javascript |
| **Database** | MySQL/MariaDB |
| **IDE and JDK** | Eclipse and Java 17 |
| **Application Type** | Single-page application in one HTML file |
| **Web Server** | Apache Tomcat 9 |
| **Server Environment** | XAMPP |

# CHAPTER III
# SYSTEM DESIGN

# CHAPTER III

## SYSTEM DESIGN

The QueueLess system follows a web-based client-server architecture. Students and administrators access the application through a web browser. The requests are handled by the Java Servlets running on the Apache Tomcat server. JSP pages are used to display the application interface, while Java and Servlets manage the application logic and user requests.

The application communicates with the MySQL/MariaDB database through JDBC. The database stores user information, menu items, orders, and order-item details. When a student places an order, the system processes the selected items, calculates the total amount, generates a token number, and stores the order information in the database. The administrator can retrieve the stored orders and update their status through the kitchen dashboard.

## 3.2 SYSTEM MODULES

The QueueLess system consists of the following major modules:

- **User Authentication Module** – Handles student registration, login, and role-based access.
- **Menu Module** – Displays available food items, categories, prices, and availability.
- **Order Module** – Allows students to select food items, quantities, and pickup slots.
- **Token Management Module** – Generates and displays unique tokens for placed orders.
- **Order Tracking Module** – Allows students to view their active and previous orders.
- **Kitchen Management Module** – Allows the administrator to view and update today's orders.
- **Database Module** – Stores and retrieves users, menu items, orders, and order-item information.

## 3.1 DATABASE DESIGN

The QueueLess system uses a relational database named **queueless** to store and manage information related to users, menu items, orders, and order details. The database is connected to the Java application using JDBC. The `users` table stores student and administrator details, while the `menu_items` table stores the available food items, categories, prices, and availability. The `orders` table stores order information such as the user, token number, order date, pickup slot, total amount, and order status.

The `order_items` table stores the individual food items included in each order along with their quantity and price. The tables are connected using appropriate relationships, where users can place multiple orders, each order can contain multiple order items, and each order item refers to a menu item. This database structure helps QueueLess maintain organized and consistent information and supports the complete food ordering, token generation, and order tracking process.

### 3.1.1 ER DIAGRAM

The Entity Relationship (ER) diagram of the QueueLess system represents the relationships between the main entities involved in the food ordering process. The major entities are **Users, Menu Items, Orders, and Order Items**. The Users entity stores student and administrator information, while Menu Items stores details of the food available in the canteen. The Orders entity maintains information such as token number, order date, pickup slot, total amount, and order status.

The relationships between these entities represent the complete ordering process. A user can place multiple orders, while each order can contain multiple order items. Each order item is associated with a particular menu item and stores its quantity and price. These relationships allow the QueueLess system to maintain organized information about users, food items, and orders and support efficient token generation and order tracking.

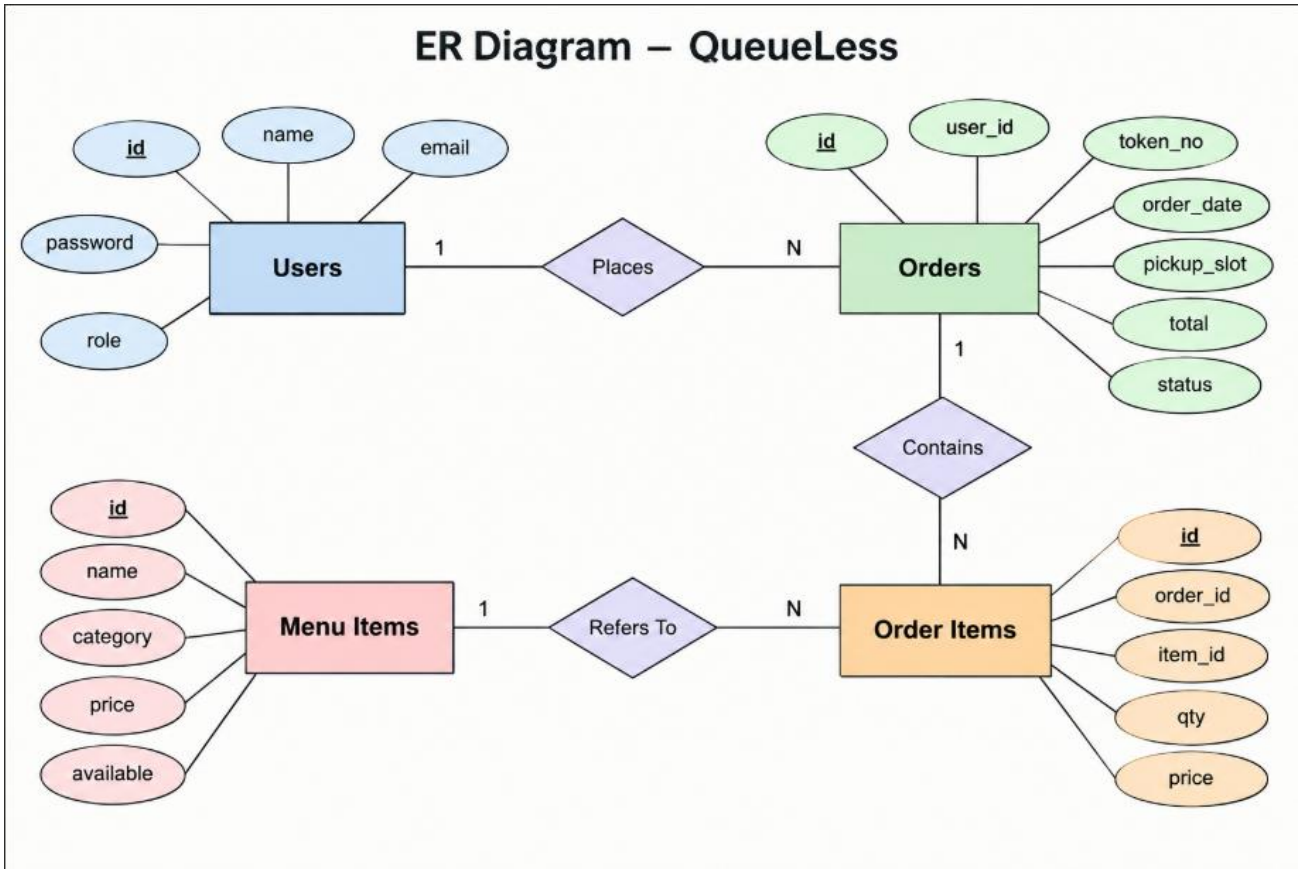



Fig 3.1 Entity Relationship Diagram of the QueueLESS web-based ordering

### 3.1.2 NORMALIZATION

Normalization is the process of organizing the QueueLess database to reduce data redundancy and improve data consistency. The system separates information into different tables such as `users`, `menu_items`, `orders`, and `order_items`. Each table stores information related to a specific entity, which avoids unnecessary duplication and makes the database easier to manage.

The database follows the basic principles of normalization by ensuring that each field contains a single value and that each record can be uniquely identified using a primary key. The `users` table stores user information, while the `menu_items` table stores food item details. The `orders` table stores the main order information, and the `order_items` table stores the individual items and quantities associated with each order.

The relationships between these tables are maintained using foreign keys such as `user_id`, `order_id`, and `item_id`. This structure helps maintain referential integrity and prevents inconsistent or repeated data. Therefore, normalization in QueueLess provides an organized database structure that supports efficient storage, retrieval, and management of food ordering and token information.

| **USERS** | | | | |
|---|---|---|---|---|
| **USER_ID** | **NAME** | **EMAIL** | **PASS** | **ROLE** |
| 101 | Ram | abc@gmail.com | ****** | ADMIN |
| 102 | John | Xyz@gmail.com | ****** | STUDENT |

| **MENU_ITEMS** | | | | |
|---|---|---|---|---|
| **ITEM_ID** | **NAME** | **CATEGORY** | **PRICE** | **AVAILABLE** |
| 1 | Chicken briyani | Main course | 110.00 | 2 |
| 2 | Veg Fried Rice | Main course | 80.00 | 1 |
| 3 | Tea | Beverage | 15.00 | 1 |

| ORDER_ID | USER_ID | TOKEN_NO | ORDER_DATE | PICKUP_SLOT | TOTAL | STATUS |
|---|---|---|---|---|---|---|
| 1 | 2 | 1 | 06-10-2026 | 10:30 AM | 110.00 | PLACED |
| 2 | 2 | 2 | 06-10-2026 | 11:30 AM | 95.00 | PREPARING |

| ID | ORDER_ID | ITEM_ID | QTY | PRICE |
|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 110.00 |
| 2 | 2 | 2 | 1 | 80.00 |
| 3 | 2 | 3 | 1 | 15.00 |

The normalization process in QueueLess begins by identifying the main entities and separating their attributes into appropriate tables. The user information is stored in the `users` table, food item details are stored in the `menu_items` table, order information is stored in the `orders` table, and the individual items within each order are stored in the `order_items` table. In the first normal form (1NF), each field contains a single value and there are no repeating groups. In the second normal form (2NF), all non-key attributes depend on the complete primary key of their respective table. In the third normal form (3NF), non-key attributes depend only on the primary key and not on other non-key attributes.

After applying these normalization steps, the QueueLess database avoids unnecessary duplication and maintains consistent data. The `users` table uses `id` as its primary key, `menu_items` uses `id`, `orders` uses `id`, and `order_items` uses its own `id` while maintaining relationships through `user_id`, `order_id`, and `item_id`. This separation ensures that changes to user, menu, or order information can be made independently without creating duplicate records. The final normalized structure improves data integrity, reduces redundancy, and provides efficient storage and retrieval of information for the QueueLess food ordering and token management system.

### 3.1.3 DATA DICTIONARY

A data dictionary, also known as a metadata repository, is a centralized collection of information about the data used in a database. It provides details about the meaning, structure, format, relationships, and usage of different data elements. In the QueueLess system, the data dictionary describes the tables, fields, data types, keys, and relationships used to manage users, menu items, orders, and order details.

The data dictionary of QueueLess consists of four main tables: `users`, `menu_items`, `orders`, and `order_items`. The `users` table stores the login and role information of students and administrators. The `menu_items` table stores the food items available in the canteen along with their category, price, and availability. The `orders` table stores the main details of each order, while the `order_items` table stores the individual food items, quantities, and prices associated with each order.

The data dictionary helps maintain consistency in the database by defining the names and characteristics of each field. Primary keys uniquely identify records, while foreign keys establish relationships between related tables. By clearly defining the structure and purpose of each field, the data dictionary makes the QueueLess database easier to understand, maintain, and develop.

Table: Student

| S.NO | COLUMN NAME | DATA TYPE | LENGTH | DESCRIPTION |
|---|---|---|---|---|
| 1 | Id | Number | 11 | User identification number (Primary Key) |
| 2 | Name | Varchar | 100 | Name of the student or administrator |
| 3 | Email | Varchar | 150 | Email address used for login |
| 4 | Password | Varchar | 250 | User password stored as a hash |
| 5 | Role | Varchar | 20 | Cumulative grade point average (0 to 10) |

Table: Menu_Items

| S.NO | COLUMN NAME | DATA TYPE | LENGTH | DESCRIPTION |
|---|---|---|---|---|
| 1 | Id | Number | 11 | Menu item identification number (Primary Key) |
| 2 | Name | Varchar | 150 | Name of the food item |
| 3 | Category | Varchar | 255 | Category of the food item |
| 4 | Price | Decimal | 20 | Price of the food items |
| 5 | Available | Number | 11 | Indicates whether the food items is available |

Table: Orders

| S.NO | COLUMN NAME | DATA TYPE | LENGTH | DESCRIPTION |
|---|---|---|---|---|
| 1 | User_id | Number | 11 | Order identification number (Primary Key) |
| 2 | id | Number | 10 | User who placed the order (Foreign Key) |
| 3 | Token_no | Number | 20 | Token number generated for the order |
| 4 | Order_date | Date | --- | Date on which the order was placed |
| 5 | Pickup_slot | Varchar | 50 | Selected food pickup time |
| 6 | total | Decimal | 10,2 | Total amount of the order |
| 7 | Status | Varchar/Enum | 20 | Current order status such as PLACED, PREPARING, READY, COMPLETED, or CANCELLED |

Table: Order_items

| S.NO | COLUMN NAME | DATA TYPE | LENGTH | DESCRIPTION |
|---|---|---|---|---|
| 1 | id | Number | 11 | Order item identification number (Primary Key) |
| 2 | order_id | Number | 11 | Related order identification number (Foreign Key) |
| 3 | Item_id | Number | 11 | Related menu item identification number (Foreign Key) |
| 4 | Qty | Number | 11 | Quantity of the selected food item |
| 5 | Price | Decimal | 10,2 | Price of the food item at the time of ordering |

## 3.2 HIGH LEVEL DESIGN

A **High Level Design** provides an overview of the **QueueLess** system and explains the major modules involved in the application. It describes how students and administrators interact with the system and how the different modules work together to manage food ordering, token generation, and order preparation. The project consists of the following modules:

**Authentication and Role Module:**

The login page allows users to sign in using their registered email and password. The system identifies the user's role as Student or Admin and provides access to the appropriate interface. Students are directed to the menu and order-related features, while administrators are provided with access to the kitchen dashboard. The authentication module ensures that users can access only the functions available for their respective roles.

**Menu Browsing Module:**

In the menu browsing module, students can view the available food items displayed with their name, category, price, and availability. Students can select the required food items and specify the quantity using the quantity controls. The system calculates the total amount based on the selected items and allows students to proceed with the order after selecting the required food items.

**Order Placement and Token Module:**

In this module, students can select their preferred pickup slot and place the food order. The system validates the selected items and calculates the total order amount before creating the order. After successful order placement, a unique token number is generated for the day's orders and stored along with the order details. The student can then view the token and pickup information through the order section.

**Order Tracking and Cancellation Module:**

In the order tracking module, students can view their active and previous orders with details such as token number, ordered items, pickup time, total amount, order date, and current status. The order status is updated according to the preparation stage, such as Placed, Preparing, Ready, or Completed. Students can also cancel an eligible order, after which the order is displayed with the Cancelled status.

**Kitchen and Admin Management Module:**

In the management module, administrators can view the day's orders through the kitchen dashboard. Each order displays the token number, student name, ordered items, pickup time, total amount, and current status. The administrator can update the order status as the food is being prepared and made ready for pickup. This module helps the canteen administrator monitor and manage multiple orders in an organized manner.

## 3.3 DETAILED DESIGN

The detailed design describes the internal functions of the QueueLess system and explains how each module performs its operations. The system is divided into Authentication, Menu Browsing, Order and Token Management, Order Tracking and Cancellation, and Kitchen and Admin Management modules. Each module contains functions that receive required parameters, perform the specified operation, and return the appropriate result or response.

# 3.3.1 AUTHENTICATION AND ROLE MODULE

| Module Name | Function | Parameters | Return Value | Description | Calling Function | Function Called By |
|---|---|---|---|---|---|---|
| Authentication | Login | email, password | User session / Login result | Validates the user's credentials and creates a session after successful authentication. | Login page | Authentication process |
| Authentication | Password Hashing | password | Hashed password | Converts the entered password into a secure hash for comparison with the stored password. | Login | Login |
| Authentication | Role Validation | role | Student/Admin access | Identifies the user's role and provides access to the corresponding module. | Login | Dashboard/Menu |
| Authentication | Logout | Session | Login page | Terminates the current user session and redirects the user to the login page. | Navigation | Authentication |

# 3.3.2 MENU BROWSING MODULE

| Module Name | Function | Parameters | Return Value | Description | Calling Function | Function Called |
|---|---|---|---|---|---|---|

| | | | | | | By |
|---|---|---|---|---|---|---|
| Menu | Display Menu | User session | Menu items | Retrieves the available food items and displays them to the student. | Student dashboard | Menu module |
| Menu | Get Menu Items | Availability | List of menu items | Retrieves food item name, category, price, and availability from the database. | Display Menu | Database |
| Menu | Select Item | Item ID, quantity | Selected item | Adds the selected food item and quantity to the student's order selection. | Menu page | Order module |
| Menu | Calculate Total | Item price, quantity | Total amount | Calculates the total amount based on the selected food items and quantities. | Menu page | Order module |
| Menu | Select Pickup Slot | Pickup slot | Selected slot | Stores the pickup time selected by the student before placing the order. | Menu page | Order module |

## 3.3.3 ORDER PLACEMENT AND TOKEN MODULE

| Module Name | Function | Parameters | Return Value | Description | Calling Function | Function Called By |
|---|---|---|---|---|---|---|
| Order | Validate Order | User ID, items, quantity | Validation result | Checks whether the student has selected at least one available food item before placing the order. | Menu | Order placement |
| Order | Calculate Order Total | Item ID, quantity, price | Total amount | Calculates the final amount for all selected items. | Order placement | Database |
| Order | Generate Token | Order date | Token number | Generates the next available token number for the current day's orders. | Order placement | Order module |
| Order | Create Order | User ID, token, pickup slot, total | Order ID | Creates and stores the main order information in the database. | Place Order | Database |
| Order | Create Order Items | Order ID, item ID, quantity, | Order item records | Stores each selected food item belonging to the order. | Create Order | Database |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | price | | | | |
| Order | Place Order | User ID, selected items, pickup slot | Order confirmation | Processes the order, generates a token, stores the order details, and redirects the student to the order page. | Menu | Order module |

## 3.3.4 ORDER TRACKING AND CANCELLATION MODULE

| Module Name | Function | Parameters | Return Value | Description | Calling Function | Function Called By |
|---|---|---|---|---|---|---|
| Order Tracking | View Orders | User ID | List of orders | Retrieves the student's active and previous orders from the database. | My Tokens | Database |
| Order Tracking | Display Token | Order ID | Token details | Displays the token number and related order information to the student. | My Tokens | Order Tracking |
| Order Tracking | Display Order Status | Order ID | Current status | Displays the current preparation status of the student's order. | My Tokens | Order Tracking |
| Cancellation | Cancel Order | Order ID, User ID | Updated order status | Changes an eligible order status to CANCELLED. | Cancel Token | Database |
| Cancellation | Validate Cancellation | Order status | Validation result | Checks whether the order is eligible for cancellation before updating its status. | Cancel Order | Cancellation module |

## 3.3.5 KITCHEN AND ADMIN MANAGEMENT MODULE

| Module Name | Function | Parameters | Return Value | Description | Calling Function | Function Called By |
|---|---|---|---|---|---|---|

| | | | | | | |
|---|---|---|---|---|---|---|
| Admin | View Today's Orders | Current date | List of orders | Retrieves the orders placed on the current date for the kitchen dashboard. | Admin dashboard | Database |
| Admin | Display Order Details | Order ID | Order details | Displays token number, student name, items, pickup time, total, and status. | Today's Orders | Admin module |
| Admin | Update Order Status | Order ID, status | Updated status | Updates the preparation status of an order in the database. | Kitchen dashboard | Database |
| Admin | Manage Order | Order ID | Updated order | Allows the administrator to monitor and update the order during preparation. | Admin dashboard | Order Management |
| Admin | View Token Information | Order ID | Token details | Displays the token number associated with the selected order. | Today's Orders | Admin module |

## 3.3.6 DATABASE MODULE

| Module Name | Function | Parameters | Return Value | Description | Calling Function | Function Called By |
|---|---|---|---|---|---|---|
| Database | Get Connection | Database configuration | Connection object | Establishes a JDBC connection between the QueueLess application and the database. | Servlet / Application | Database operations |
| Database | Query | SQL query, parameters | Result records | Executes SELECT operations and retrieves required information from the database. | Application modules | Database |
| Database | Update | SQL query, parameters | Number of affected rows | Executes INSERT, UPDATE, or DELETE operations on the database. | Application modules | Database |

| | | | | | | |
|---|---|---|---|---|---|---|
| Database | Close Connection | Connection object | None | Closes the database connection after completing the required operation. | Database operations | Database |

## 3.3.7 MODULE FLOW

The main flow of the QueueLess system starts with the **Authentication and Role Module**. After successful login, a student is directed to the **Menu Browsing Module**, where available food items are retrieved from the database. The student selects food items, quantities, and a pickup slot, after which the **Order Placement and Token Module** validates the selection, calculates the total, generates a token, and stores the order.

The student can then use the **Order Tracking and Cancellation Module** to view the token, order details, and current status or cancel an eligible order. At the same time, the administrator accesses the **Kitchen and Admin Management Module** to view today's orders and update their preparation status. All these modules communicate with the **Database Module**, which provides the required database connection and performs the necessary data retrieval and modification operations.

# CHAPTER IV

# IMPLEMENTATION

# CHAPTER IV

## IMPLEMENTATION

The third step of software development is implementation and coding. Implementation is the stage in which the designed QueueLess system is converted into an actual working web application. After completing the system design, the major functions, modules, database structure, and user interactions are defined. The main objective of the implementation phase is to translate the design of the QueueLess food ordering and token management system into functional program code using appropriate web technologies.

The coding phase of this project is implemented using Java, JSP, Java Servlets, HTML, CSS, JavaScript, JDBC, and MySQL/MariaDB. JSP pages are used to create the user interface, while Java Servlets handle requests and application logic. JDBC provides communication between the Java application and the database. Apache Tomcat is used as the web server for running the application, while XAMPP provides the database service and phpMyAdmin is used for database management. The major modules such as login, menu, order placement, token generation, order tracking, cancellation, and administrator order management are implemented according to the system design.

The implementation phase has an important effect on testing and maintenance. Well-structured and simple code reduces errors and makes the application easier to test, understand, and maintain. Therefore, the coding of QueueLess focuses on simplicity, modularity, database consistency, and proper separation of user and administrator functions. After the coding is completed, the application is deployed on the Apache Tomcat server and accessed through a web browser. Each module is then tested to verify that it performs the required operation correctly.

## 4.1 DEPLOYMENT PROCEDURE

The deployment procedure describes the steps required to prepare the environment, configure the database and server, deploy the QueueLess application, and access it through a web browser.

### Preparing the Project

The QueueLess project is developed using Java, JSP, Servlets, HTML, CSS, and JavaScript. Eclipse IDE is used for development and project management. Java Development Kit 17 is installed and configured for compiling and running the Java application. Apache Tomcat 9 is configured in Eclipse as the application server.

The project is maintained with the required JSP files, Java Servlet classes, CSS files, JavaScript files, and configuration files. The project uses the web application context named **QueueLess**.

### Preparing the Database

The database service is started using XAMPP. The MySQL/MariaDB service provides the database required by the application. The database named **queueless** is created using phpMyAdmin.

The database contains the required tables for storing application data. The main tables are `users`, `menu_items`, `orders`, and `order_items`. User information, food items, order information, and individual order items are stored in these tables.

## Configuring Database Connectivity

The QueueLess application uses JDBC to communicate with the database. The database connection is configured in the application's database utility class. The JDBC connection provides the required connection between the Java Servlets and the `queueless` database.

When a user performs an operation that requires database information, the Servlet communicates with the database through JDBC. The required records are retrieved or updated and the result is then displayed through the appropriate JSP page.

## Starting the Application Server

After configuring the project and database, the Apache Tomcat 9 server is started through Eclipse. The server runs the QueueLess web application and processes requests received from the browser. The application can then be accessed using the local web address:
**http://localhost:8080/QueueLess/**
The browser sends requests to the Tomcat server, which processes the request through the appropriate Servlet or JSP page and returns the required response.

## Running the Application

The complete application execution requires the following services:

1. Start the Apache service from XAMPP.
2. Start the MySQL/MariaDB service from XAMPP.
3. Start Apache Tomcat 9 from Eclipse.
4. Open a web browser.
5. Enter the QueueLess application address.
6. Log in using the appropriate student or administrator account.
7. Test the required functions of the application.

The student can log in, view the menu, select food items, choose quantities and a pickup slot, place an order, receive a token, and track the order. The administrator can log in to the kitchen dashboard and manage the day's orders.

## Stopping the Application

After completing the testing or demonstration, the Apache Tomcat server can be stopped from Eclipse. The Apache and MySQL/MariaDB services can then be stopped from XAMPP. This ensures that the application server and database services are properly terminated.

# 4.2 CODING

Coding is one of the important parts of the project in which the QueueLess system is developed according to the requirements of the users. The coding process converts the system design into a working application using Java, JSP, Servlets, HTML, CSS, JavaScript, and JDBC. The code is

organized into separate presentation, servlet, utility, and database-related components so that each part of the application performs a specific function.

## JSP and Front-End Implementation

JSP pages are used to create the user interface of QueueLess. The pages provide forms, navigation elements, food menu cards, order information, token information, and administrator dashboard components. HTML provides the basic structure of the pages, CSS is used for styling and layout, and JavaScript is used for client-side interactions such as quantity selection and total amount calculation.

The login page provides the authentication interface for students and administrators. The menu page displays available food items and allows students to select the required quantity. The order page displays token and order information, while the administrator page displays the day's orders and their current status.

## Servlet Implementation

Java Servlets are used to process user requests and implement the main application logic. The Servlets receive requests from JSP pages, validate the required information, communicate with the database, and forward or redirect the user to the appropriate page.

The authentication process validates the entered login credentials and identifies the user's role. The menu and order processes retrieve available food items, calculate order information, generate tokens, and store order records. The order management process allows the administrator to retrieve orders and update their preparation status.

## Database Connectivity Implementation

JDBC is used to establish communication between the Java application and the MySQL/MariaDB database. The database utility class provides the connection required by the Servlets to execute SQL operations.

The application performs database operations such as insertion, retrieval, updating, and deletion. User information is retrieved during authentication, menu items are retrieved for display, orders and order items are inserted when a student places an order, and order status is updated when the administrator changes the preparation stage.

## Authentication Implementation

The authentication module validates the email and password entered by the user. The system retrieves the corresponding user information from the `users` table and checks the credentials. After successful authentication, the user's name, user ID, and role are maintained in the session.

Based on the role, the system provides the appropriate access. Students can access the menu and their orders, while administrators can access the kitchen management dashboard.

## Menu Implementation

The menu module retrieves available food items from the `menu_items` table. Each available item is displayed with its name, category, price, and availability information.Students can select the required quantity for each available item. The selected quantities are processed when the order is submitted, and the total amount is calculated based on the price and quantity of each selected item.

## Order and Token Implementation

The order module processes the food items selected by the student and validates that at least one item has been selected. The system calculates the total amount and retrieves the next available token number for the current date.

The order information is stored in the `orders` table, while the individual food items are stored in the `order_items` table. The order is stored with the selected pickup slot and an initial status of **PLACED**. After successful insertion, the student is redirected to the order page where the generated token can be viewed.

## Order Tracking Implementation

The order tracking module allows students to view their active and previous orders. The system retrieves order information based on the logged-in user's ID.

The displayed information includes the token number, ordered items, pickup slot, total amount, order date, and current status. This allows students to monitor the progress of their orders without repeatedly approaching the canteen counter.

## Order Cancellation Implementation

The cancellation function allows students to cancel eligible orders. When the cancellation request is submitted, the system identifies the selected order and verifies that it belongs to the logged-in student.

After successful validation, the order status is changed to **CANCELLED** in the database. The updated status is then displayed in the student's order list. Orders that have already reached the final preparation stages are not treated as normally cancellable

## Administrator Implementation

The administrator module provides a kitchen dashboard for managing the day's orders. The system retrieves orders placed on the current date and displays important information such as token number, student name, ordered items, pickup time, total amount, and status.

The administrator can update the status of an order according to its preparation stage. The supported statuses include **PLACED, PREPARING, READY, COMPLETED, and CANCELLED**. These changes are stored in the database and become visible to the student through the order tracking page.

## Session Management

Session management is used to maintain information about the currently logged-in user. The user's ID, name, and role are stored in the session after successful authentication.

The session information is used to restrict access to protected pages and to retrieve the orders belonging to the logged-in student. When the user selects Logout, the session is terminated and the user is returned to the login page.

## Database Operations

The QueueLess application performs four major types of database operations:

- **Insertion** – Used to add new users, orders, and order-item records.

- **Retrieval** – Used to display users, available menu items, student orders, and administrator orders.
- **Updation** – Used to update order status and other required information.
- **Deletion** – Used where records need to be removed according to the application's requirements.

These operations allow the application to maintain consistent and up-to-date information throughout the ordering process.

## 1. Database Connection — `DB.java`

```
public class DB {

    private static final String URL =
        "jdbc:mysql://localhost:3306/queueless";

    private static final String USER = "root";
    private static final String PASSWORD = "root";

    public static Connection get() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

## 2. Login — `LoginServlet.java`

```
String email = request.getParameter("email");
String password = request.getParameter("password");

String sql = "SELECT * FROM users WHERE email=? AND password=?";

PreparedStatement ps = DB.get().prepareStatement(sql);
ps.setString(1, email);
ps.setString(2, hashPassword(password));

ResultSet rs = ps.executeQuery();

if (rs.next()) {
    HttpSession session = request.getSession();

    session.setAttribute("uid", rs.getInt("id"));
    session.setAttribute("name", rs.getString("name"));
    session.setAttribute("role", rs.getString("role"));

    if ("ADMIN".equals(rs.getString("role")))
        response.sendRedirect("admin");
    else
        response.sendRedirect("menu");}
```

### 3. Display Menu — `MenuServlet.java`

```
String sql = "SELECT * FROM menu_items WHERE available=1";

List<Map<String,Object>> items = DB.query(sql);

request.setAttribute("items", items);

request.getRequestDispatcher("menu.jsp")
       .forward(request, response);
```

### 4. Place Order & Generate Token — `OrderServlet.java`

This is one of the **most important snippets** for your project:

```
int token;
String sql ="SELECT COALESCE(MAX(token_no),0)+1 " +
    "FROM orders WHERE order_date=CURDATE() FOR UPDATE";
try (Statement st = connection.createStatement();
     ResultSet rs = st.executeQuery(sql)) {
    rs.next();
    token = rs.getInt(1);
}
PreparedStatement ps = connection.prepareStatement(
    "INSERT INTO orders " +
    "(user_id, token_no, order_date, pickup_slot, total) " +
    "VALUES (?, ?, CURDATE(), ?, ?)");

ps.setObject(1, userId);
ps.setInt(2, token);
ps.setString(3, pickupSlot);
ps.setDouble(4, total);
ps.executeUpdate();
```

### 5. Store Order Items

```
PreparedStatement ps = connection.prepareStatement(
    "INSERT INTO order_items " +
    "(order_id, item_id, qty, price) VALUES (?, ?, ?, ?)");
ps.setLong(1,orderId);
ps.setInt(2,itemId);
ps.setInt(3,quantity);
ps.setDouble(4,price);
ps.executeUpdate();
```

### 6. Student Order Tracking

```
String sql =
    "SELECT o.token_no, o.pickup_slot, o.total, o.status, " +
    "o.order_date, " +
    "GROUP_CONCAT(CONCAT(i.name,' x',oi.qty)) AS items " +
```

```
    "FROM orders o " +
    "JOIN order_items oi ON oi.order_id=o.id " +
    "JOIN menu_items i ON i.id=oi.item_id " +
    "WHERE o.user_id=? " +
    "GROUP BY o.id " +
    "ORDER BY o.id DESC";

PreparedStatement ps = DB.get().prepareStatement(sql);
ps.setObject(1, userId);

ResultSet rs = ps.executeQuery();
```

### 7. Cancel Order

```
String sql =
    "UPDATE orders SET status='CANCELLED' " +
    "WHERE id=? AND user_id=?";

PreparedStatement ps = DB.get().prepareStatement(sql);

ps.setInt(1, orderId);
ps.setObject(2, userId);

ps.executeUpdate();
```

### 8. Admin Update Order Status

```
String sql =
    "UPDATE orders SET status=? WHERE id=?";

PreparedStatement ps = DB.get().prepareStatement(sql);

ps.setString(1, status);
ps.setInt(2, orderId);

ps.executeUpdate();
```

## 4.3 SCREENSHOTS

The screenshots section presents the working implementation of the QueueLess system. The screenshots demonstrate the user interface and the execution of the major modules developed during the implementation phase.

The implementation screenshots include the login and registration interface, student menu, food item selection, pickup slot selection, order placement, generated token, student order tracking, order cancellation, administrator login, kitchen dashboard, and order status management.

Each screenshot represents a particular stage of the QueueLess workflow and provides visual evidence that the corresponding module has been implemented and tested successfully. The screenshots also demonstrate the interaction between the JSP interface, Java Servlets, database operations, and the Apache Tomcat server.

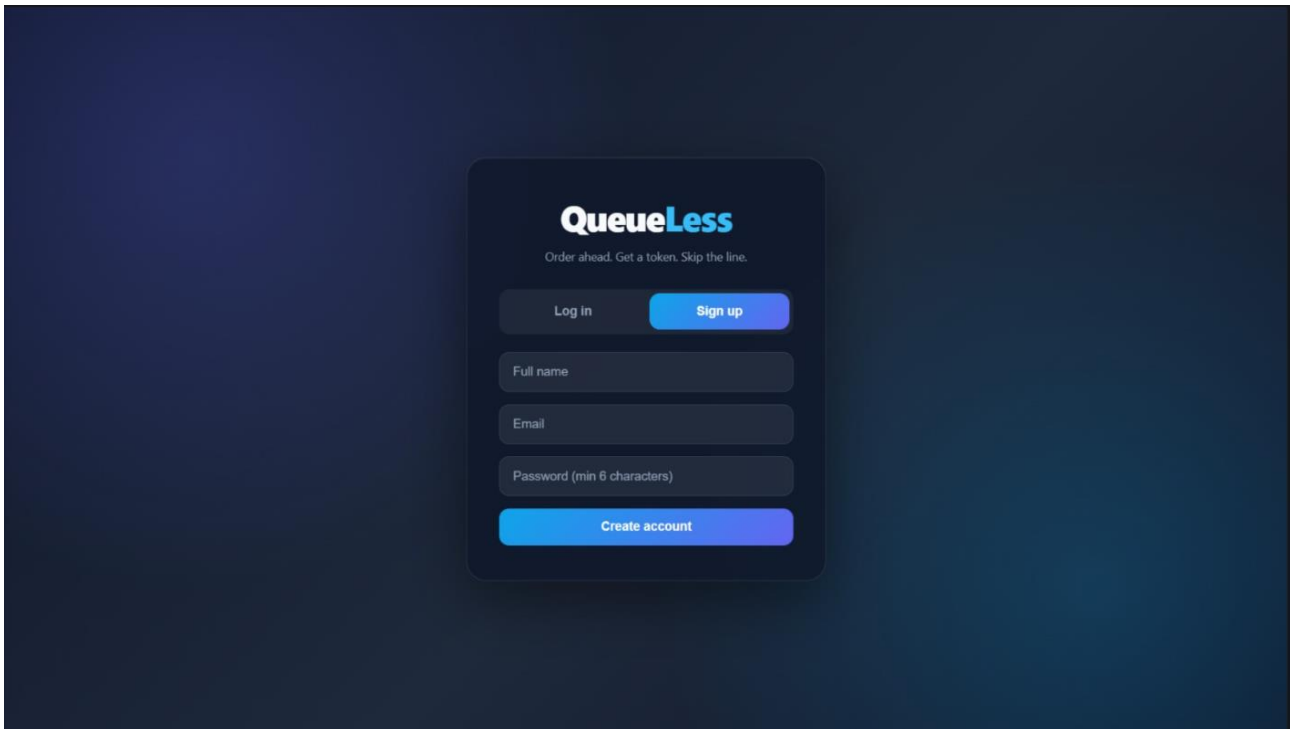

QueueLess
Order ahead. Get a token. Skip the line.
Log in
Sign up
Full name
Email
Password (min 6 characters)
Create account

Fig 4.1

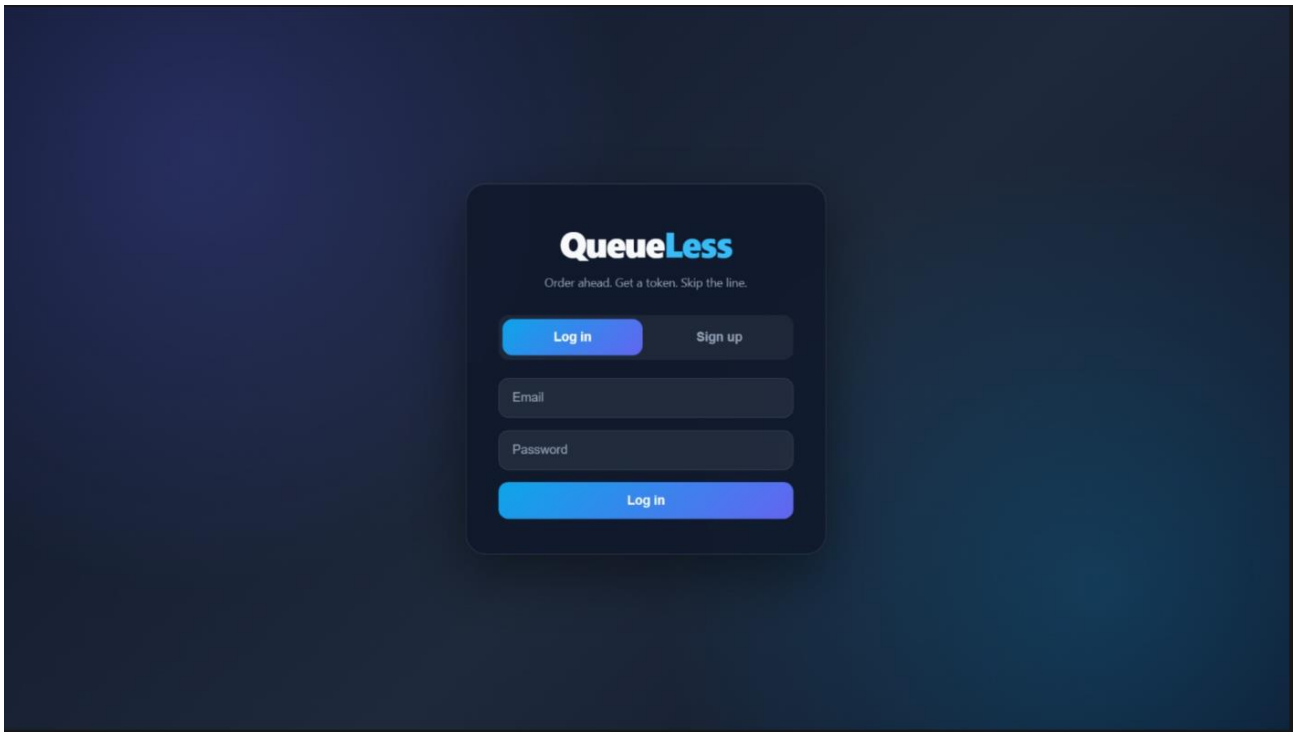

QueueLess
Order ahead. Get a token. Skip the line.
Log in
Sign up
Email
Password
Log in

Fig 4.2

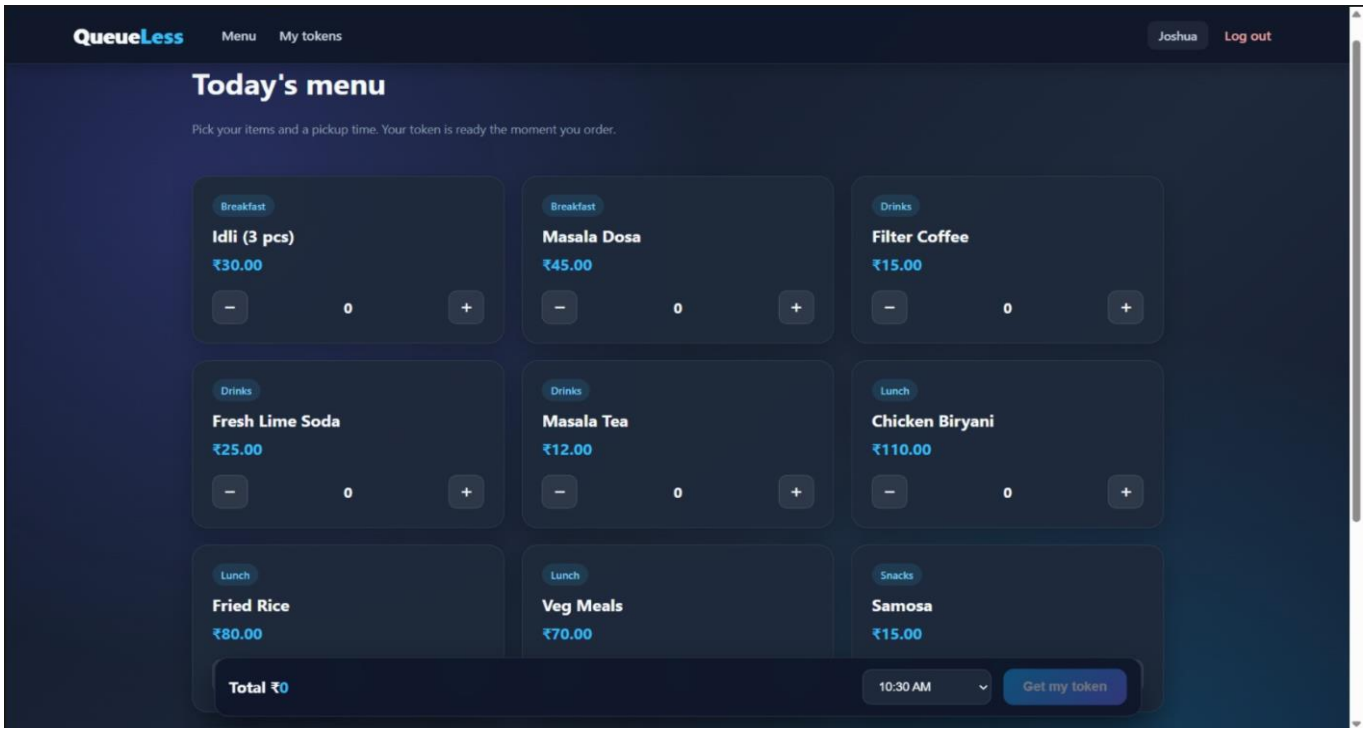

QueueLess
Menu
My tokens
Joshua
Log out
Today's menu
Pick your items and a pickup time. Your token is ready the moment you order.
Breakfast
Idli (3 pcs)
₹30.00
Breakfast
Masala Dosa
₹45.00
Drinks
Filter Coffee
₹15.00
Drinks
Fresh Lime Soda
₹25.00
Drinks
Masala Tea
₹12.00
Lunch
Chicken Biryani
₹110.00
Lunch
Fried Rice
₹80.00
Lunch
Veg Meals
₹70.00
Snacks
Samosa
₹15.00
Total ₹0
10:30 AM
Get my token

Fig 4.3

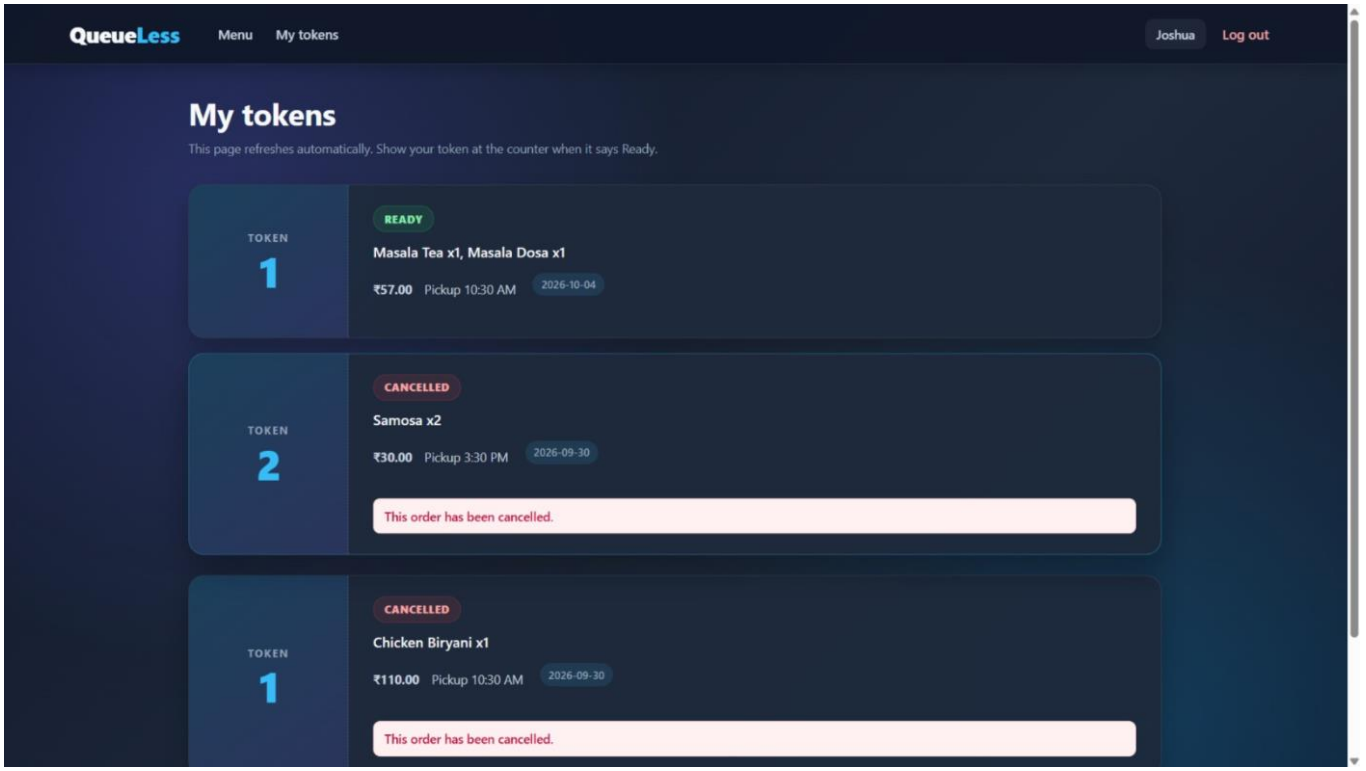

QueueLess
Menu
My tokens
Joshua
Log out
My tokens
This page refreshes automatically. Show your token at the counter when it says Ready.
TOKEN
1
READY
Masala Tea x1, Masala Dosa x1
₹57.00 Pickup 10:30 AM
2026-10-04
TOKEN
2
CANCELLED
Samosa x2
₹30.00 Pickup 3:30 PM
2026-09-30
This order has been cancelled.
TOKEN
1
CANCELLED
Chicken Biryani x1
₹110.00 Pickup 10:30 AM
2026-09-30
This order has been cancelled.

Fig 4.4

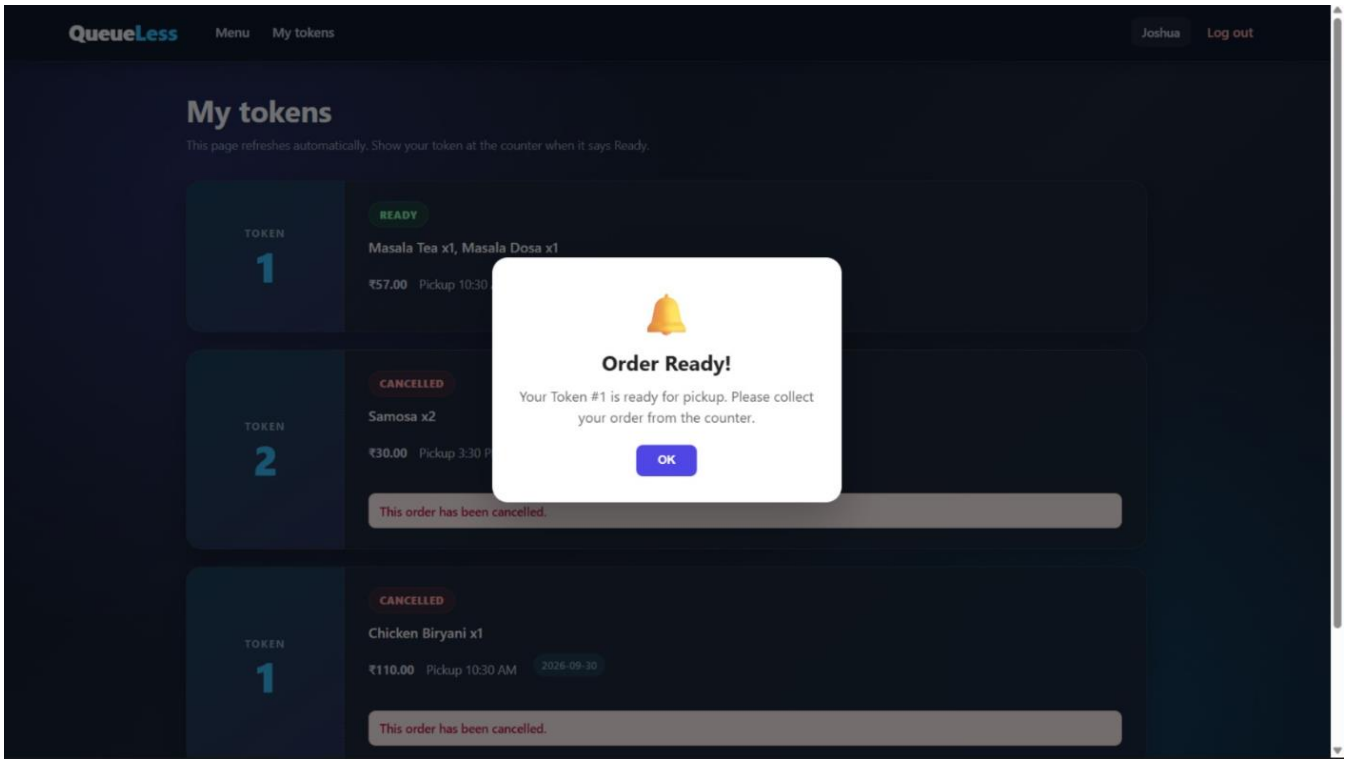

QueueLess
Menu
My tokens
Joshua
Log out
My tokens
This page refreshes automatically. Show your token at the counter when it says Ready.
TOKEN
1
READY
Masala Tea x1, Masala Dosa x1
₹57.00
Order Ready!
Your Token #1 is ready for pickup. Please collect your order from the counter.
OK
TOKEN
2
CANCELLED
Samosa x2
₹30.00
This order has been cancelled.
TOKEN
1
CANCELLED
Chicken Biryani x1
₹110.00
Pickup 10:30 AM
2026-09-30
This order has been cancelled.

Fig 4.5

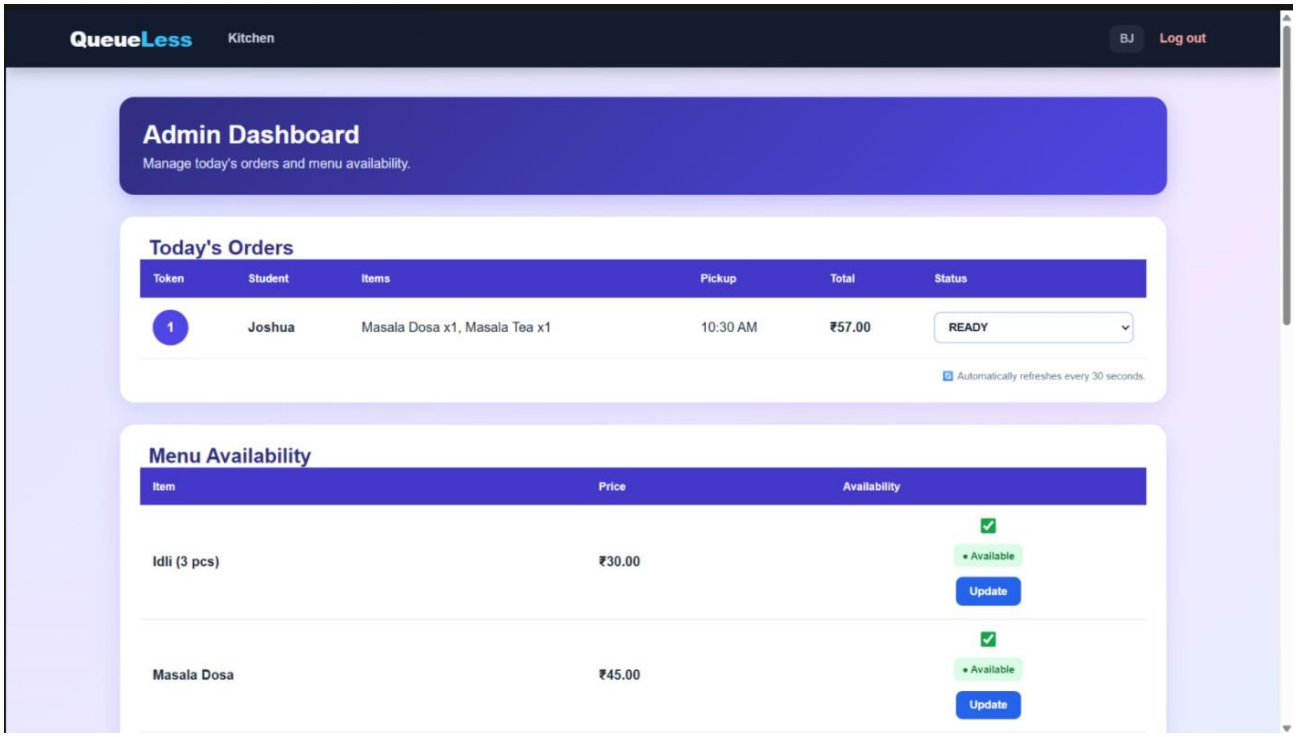

QueueLess
Kitchen
BJ
Log out
Admin Dashboard
Manage today's orders and menu availability.
Today's Orders
Token
Student
Items
Pickup
Total
Status
1
Joshua
Masala Dosa x1, Masala Tea x1
10:30 AM
₹57.00
READY
Automatically refreshes every 30 seconds.
Menu Availability
Item
Price
Availability
Idli (3 pcs)
₹30.00
Available
Update
Masala Dosa
₹45.00
Available
Update

Fig 4.6

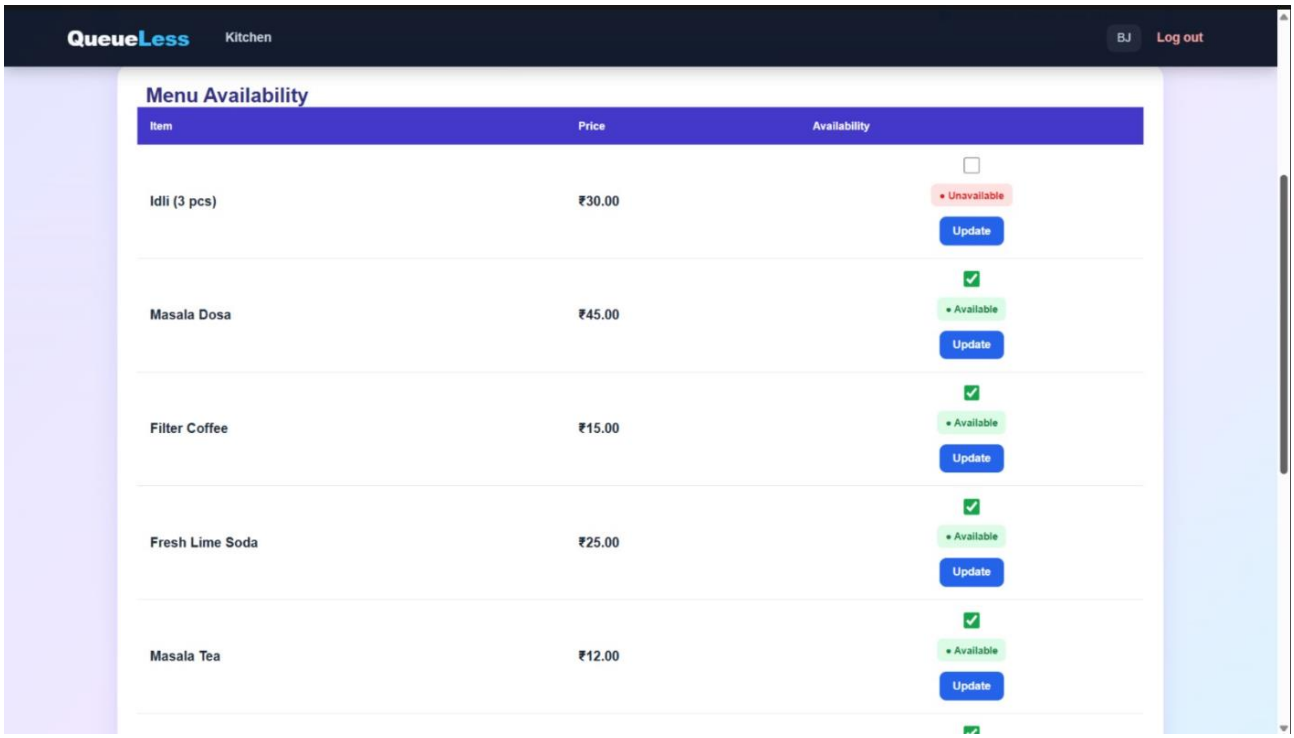

QueueLess
Kitchen
BJ
Log out
Menu Availability
Item
Price
Availability
Idli (3 pcs)
₹30.00
• Unavailable
Update
Masala Dosa
₹45.00
• Available
Update
Filter Coffee
₹15.00
• Available
Update
Fresh Lime Soda
₹25.00
• Available
Update
Masala Tea
₹12.00
• Available
Update

Fig 4.7

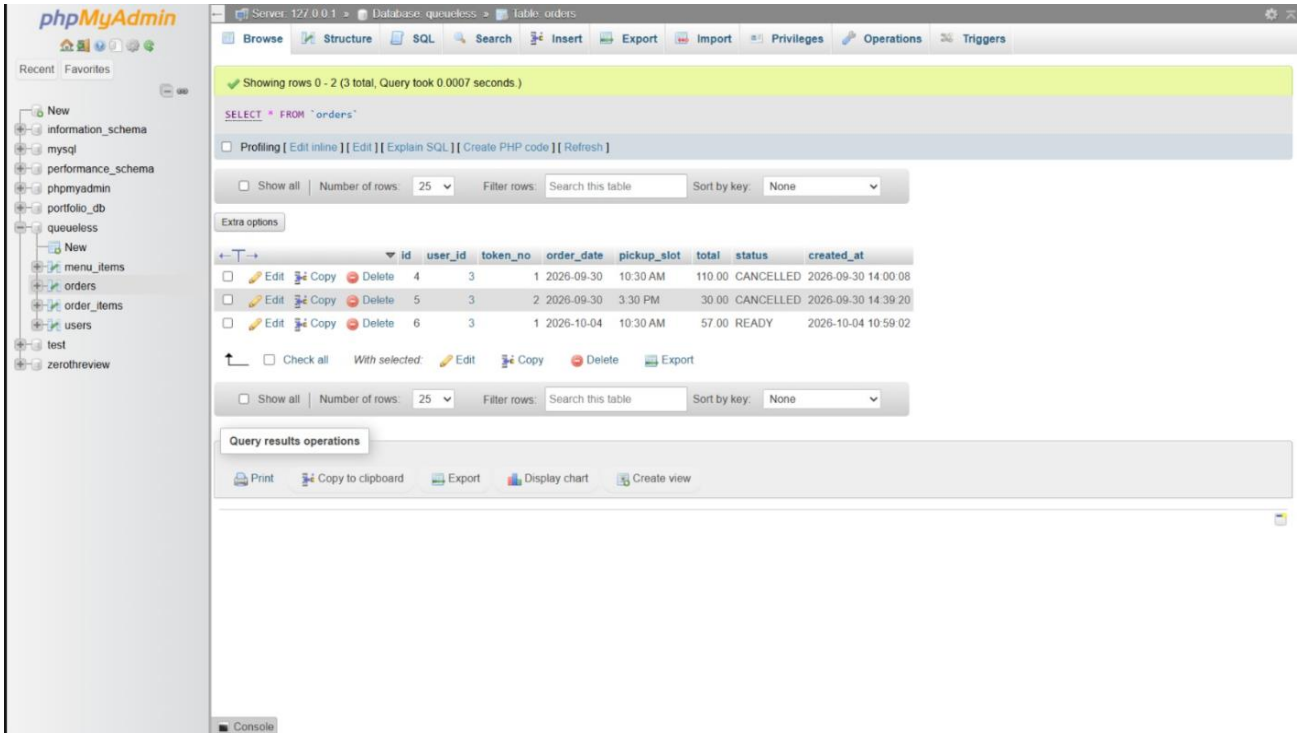

phpMyAdmin
Recent
Favorites
New
information_schema
mysql
performance_schema
phpmyadmin
portfolio_db
queueless
New
menu_items
orders
order_items
users
test
zerothreview
Server: 127.0.0.1 » Database: queueless » Table: orders
Browse
Structure
SQL
Search
Insert
Export
Import
Privileges
Operations
Triggers
Showing rows 0 - 2 (3 total, Query took 0.0007 seconds.)
SELECT * FROM `orders`
Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]
Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None
Extra options
id user_id token_no order_date pickup_slot total status created_at
Edit Copy Delete 4 3 1 2026-09-30 10:30 AM 110.00 CANCELLED 2026-09-30 14:00:08
Edit Copy Delete 5 3 2 2026-09-30 3:30 PM 30.00 CANCELLED 2026-09-30 14:39:20
Edit Copy Delete 6 3 1 2026-10-04 10:30 AM 57.00 READY 2026-10-04 10:59:02
Check all With selected: Edit Copy Delete Export
Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None
Query results operations
Print
Copy to clipboard
Export
Display chart
Create view
Console

Fig 4.8

# CHAPTER V

# CONCLUSION AND FUTURE ENHANCEMENT

# 5.1 CONCLUSION

The QueueLess system provides a simple and organized solution for managing food ordering and token generation in a college canteen. The system allows students to view the available menu, select food items, choose quantities and pickup slots, place orders, and receive a unique token number. Students can also view their order details and track the current status of their orders through the application.

The system also provides a dedicated kitchen management interface for the administrator. The administrator can view the day's orders, monitor token numbers, student details, ordered items, pickup time, total amount, and update the status of orders. This helps reduce manual work and makes the food preparation and order management process more systematic.

QueueLess reduces unnecessary waiting and overcrowding by allowing students to place their orders digitally and collect them according to the selected pickup time. The use of Java, JSP, Servlets, HTML, CSS, JavaScript, JDBC, Apache Tomcat, and MySQL/MariaDB provides a suitable platform for implementing the complete food ordering and token management process.

Overall, the project successfully demonstrates how a web-based application can improve the efficiency of a college canteen by providing organized ordering, token generation, order tracking, and kitchen management features.

# 5.2 FUTURE ENHANCEMENT

The QueueLess system can be further enhanced by adding additional features to improve its functionality and user experience. Online payment can be integrated so that students can pay for their orders digitally before collecting the food. Notifications can also be added to inform students when their order is being prepared or is ready for pickup.

The system can also be enhanced with a more advanced administrator dashboard containing daily sales reports, order statistics, popular food items, and menu management features. Students could be provided with additional options such as order history, favourites, and improved search and filtering of menu items.

In the future, QueueLess can be extended into a mobile application so that students can access the ordering system more conveniently. The system can also support multiple canteens or food counters within the same institution. These enhancements can make QueueLess a more complete and scalable food ordering and queue management solution.